

The Role of System Architecture in Ensuring High Availability and Fault Tolerance

Contents

1	Introduction	2
2	Hardware Architecture for HA and FT	2
3	Software Architecture for HA and FT	3
4	Integrating Hardware and Software for HA and FT	4
5	Benefits of Robust System Architecture	4
6	Conclusion	5
7	Reference Image	6

Abstract

High Availability (HA) and Fault Tolerance (FT) are fundamental strategies in designing modern IT systems, aimed at ensuring continuous operations and minimizing service disruptions. System architecture, encompassing both hardware and software design, serves as the foundation for these strategies. This report examines how system architecture supports HA and FT by implementing redundancy, failover mechanisms, self-healing systems, and resilient network and storage designs. It highlights techniques and best practices that enable organizations to maintain business continuity, protect critical data, and recover seamlessly from failures.

1 Introduction

Modern organizations increasingly rely on IT systems to deliver essential services, manage critical data, and support business operations. Any disruption can lead to financial loss, reputational damage, or customer dissatisfaction. High Availability ensures that systems remain operational with minimal downtime, while Fault Tolerance allows them to continue functioning even when individual components fail. Achieving these goals requires a well-planned system architecture that integrates hardware and software solutions to provide redundancy, resilience, and automated recovery.

System architecture not only determines system performance but also directly influences reliability, maintainability, and operational continuity. By strategically designing servers, networks, storage, and software components, organizations can prevent failures from propagating, reduce downtime, and maintain seamless service delivery.

2 Hardware Architecture for HA and FT

Hardware architecture lays the groundwork for achieving HA and FT by introducing redundancy and resilience at the physical layer. Key components include:

Redundant Servers

Multiple servers operating in active-active or active-passive configurations ensure that system services remain available even if one server fails. Active-passive setups maintain a standby server ready to take over, while active-active configurations distribute workloads across multiple servers, eliminating single points of failure.

Network Redundancy

Reliable network connectivity is essential for HA. Dual network interfaces, redundant switches, and routers prevent downtime caused by a single point of failure. Load balancers manage traffic distribution and automatic failovers, maintaining uninterrupted access to services.

Storage Redundancy

Storage systems employ RAID configurations (RAID 1, 5, 10) to guard against disk failures, while multi-pathing provides multiple physical paths to storage devices. Hot-swappable drives enable rapid replacement without downtime, ensuring continuous access to critical data.

Power Redundancy

Servers equipped with dual power supplies, supported by Uninterruptible Power Supplies (UPS) and backup generators, continue functioning during power interruptions. This layer of redundancy protects against outages that could otherwise disrupt system availability.

In combination, these hardware strategies minimize the risk of service interruptions, creating a stable foundation for high availability and fault-tolerant operations.

3 Software Architecture for HA and FT

Software architecture complements hardware by providing logical mechanisms to maintain HA and FT. Effective software design ensures that applications and services remain available, detect errors, and recover automatically.

Redundant Software Systems

Running multiple instances of applications or services across different servers or containers ensures continuous service delivery. If one instance fails, others seamlessly handle requests, preventing downtime.

Decoupled and Modular Design

Modular or microservice architectures isolate faults within individual components, preventing failures from affecting the entire system. This approach enables graceful degradation, maintaining core functionality while issues are addressed.

Self-Healing Systems

Automated detection and recovery mechanisms allow software to identify faults, restart services, reroute traffic, or switch to backup resources without human intervention. Self-healing systems reduce recovery times and maintain service continuity.

Error Detection and Correction

Techniques such as parity checks, checksums, and error-correcting codes (ECC) ensure data integrity. By detecting and correcting errors, systems avoid corrupted operations and maintain reliable performance.

Through resilient software design, HA and FT are achieved even under heavy load or component failures, complementing the redundancy established at the hardware level.

4 Integrating Hardware and Software for HA and FT

The synergy between hardware and software architectures is crucial for maximizing system availability and fault tolerance:

- **Redundancy Across Layers:** Hardware redundancy is complemented by software redundancy, ensuring continuous service even if multiple components fail simultaneously.
- **Automated Failover:** Software continuously monitors hardware health, automatically triggering failover to backup systems when failures are detected.
- **Replication and Synchronization:** Data and system states are replicated across multiple nodes and synchronized in real-time, enabling rapid recovery without service disruption.
- **Monitoring and Control:** Continuous monitoring allows early detection of anomalies, facilitating proactive intervention before downtime occurs.

This integrated approach ensures that failures are contained, automatically corrected, and that services remain operational, providing a comprehensive HA and FT solution.

5 Benefits of Robust System Architecture

A well-designed architecture provides tangible benefits:

- **Minimized Downtime:** Redundant systems and automated failover reduce the likelihood and impact of disruptions.

- **Fault Isolation and Recovery:** Failures are contained and corrected without affecting other system components.
- **Enhanced Reliability:** Combining hardware and software resilience increases trust in system performance.
- **Scalability and Performance:** Distributed architectures and load balancing support higher demand while maintaining fault tolerance.

By addressing both the physical and logical layers, organizations can protect critical data, sustain operations, and maintain service quality.

6 Conclusion

System architecture is the cornerstone of High Availability and Fault Tolerance. Redundant hardware, resilient network and storage designs, and fault-tolerant software systems together ensure that IT infrastructures continue functioning despite failures. Integrating HA and FT strategies within the architecture allows for automated failover, self-healing, and rapid recovery, ensuring minimal downtime and sustained service delivery.

Organizations that adopt these principles enhance business continuity, protect critical data, and maintain operational efficiency. Effective system architecture not only mitigates risk but also provides a competitive advantage in technology-driven markets, enabling reliable, uninterrupted service to users and stakeholders.

References

1. F. Reghenzani, Z. Guo, and W. Fornaciari, “Software Fault Tolerance in Real-Time Systems: Identifying the Future Research Questions,” vol. 55, no. 14s, pp. 1–30, Jul. 2023, doi: <https://doi.org/10.1145/3589950>.
2. C. Luca, “High Availability, Fault Tolerance, and Disaster Recovery Strategies,” Sep. 01, 2024. <https://www.researchgate.net/publication/388527642>
3. GeeksforGeeks, “Fault Tolerance in System Design,” GeeksforGeeks, Nov. 08, 2023. <https://www.geeksforgeeks.org/system-design/fault-tolerance-in-system-design/>
4. M. Cate, “AI and Machine Learning for Dynamic Workload Orchestration,” Jul. 01, 2024. <https://www.researchgate.net/publication/388527637>

7 Reference Image

<https://doi.org/10.1145/3589950>

1 INTRODUCTION AND MOTIVATION

Hard real-time systems must satisfy both temporal and logical requirements. At the design phase, system correctness is usually proved, for the former, by a scheduling analysis and, for the latter, by formal verification or extensive testing. However, even assuming flawless design of the

This work has received funding from the European Space Agency (OSIP grant no. 4000133770/21/NL/MH/hm), ICSC National Research Center in High-Performance Computing, Big Data and Quantum Computing, and NSF (grant no. CCF-2028481).

Authors' addresses: F. Reghenzani, DEIB, Politecnico di Milano, via Ponzio 34/5, Milano, ITA, 20133, ESTEC, European Space Agency, Keplerlaan 1, Noordwijk, NLD, 2201; email: federico.reghenzani@polimi.it; Z. Guo, Department of Computer Science, North Carolina State University, 2262 EB II, 890 Oval Dr, Raleigh, North Carolina, USA, 27606; email: zguo32@ncsu.edu; W. Fornaciari, DEIB, Politecnico di Milano, via Ponzio 34/5, Milano, ITA, 20133; email: william.fornaciari@polimi.it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0360-0300/2023/07-ART306 \$15.00

<https://doi.org/10.1145/3589950>

306

ACM Computing Surveys, Vol. 55, No. 14s, Article 306. Publication date: July 2023.

system, hardware can still be the cause of violation of these constraints due to its natural susceptibility to faults. In critical systems, it is essential to guarantee a certain degree of resilience to hardware faults. No system can tolerate an unlimited amount of hardware faults: safety-critical standards dictate precise fault rates that functions performed by the system must achieve to be considered compliant. This necessary dependability can be achieved by employing hardware solutions that mitigate or tolerate the faults, for instance, by redundancy. Alternatively, software techniques can be employed with the crucial advantage of not requiring any custom hardware components. These software techniques, when applied to hardware faults, are usually under the umbrella term *Software-Implemented Hardware Fault Tolerance* (abbreviated with different acronyms, SIFT [104], SWIFT [87], or SIHFT [37]). SIFT solutions have a positive impact on hardware cost and production development. This convenience becomes a vital requirement for Commercial Off-The-Shelf (COTS) hardware in which strict fault tolerance is not a common design goal. The availability of COTS hardware led to significant trends in both the automotive and aerospace sectors over the past few years in order to reduce design costs and time to market. Even space agencies have active working groups studying how to integrate COTS hardware in critical systems [70, 80].

The computational power demand of emerging applications is increasing, and hardware manufacturers are striving to increase the computational capability of the computing platforms. However, this process is obstructed by the single-core performance barrier, mainly due to thermal and power constraints, which make increasing clock frequency difficult. Therefore, to achieve the desired performance, the hardware components are becoming increasingly smaller and implement complex features (pipelines, multicore, multilevel caches, etc.). These undaunted trends have a double effect: increasing the fault rate and making the real-time problem of formally guaranteeing the temporal requirements harder. In turn, they create a challenging environment [21]: on one hand, the increased fault rate pushes for SIFT approaches; on the other hand, the timing problem makes their use difficult. The major difficulty in using modern hardware in critical systems is computation of the Worst-Case Execution Time (WCET) of the tasks, an essential metric to perform a correct scheduling analysis and, therefore, prove the temporal correctness. A partial solution to the WCET problem is the mixed-criticality model for real-time tasks, as described later in Section 2.6.

Motivation. Software fault tolerance algorithms have a clear impact on real-time performance: the necessity to reserve computational resources and time for recovery routines makes scheduling more challenging. Several state-of-the-art fault-tolerant solutions exist, and many real-time scheduling algorithms have been developed in the last 40 years. However, analyzing the real-time characteristics of fault tolerance algorithms and improving scheduling in this direction received less attention from the research community. As we present later in this article, the opposite problem – how scheduling decisions impact fault tolerance – has been analyzed by very few works. There is a great potential for the research community to address the many open challenges on fault-tolerant real-time systems. Special focus can be given to mixed-criticality systems, an attractive future direction as also stated in the Burns and Davis survey [18]: “Although there is clear link between Fault-Tolerant and Mixed-Criticality, there has not yet been much work published that directly addresses fault-tolerant mixed criticality systems.”

Situations where fault tolerance is crucial

- **RAID (Redundant Array of Independent Disks):** In storage systems, [RAID configurations](#) distribute data across multiple disks with redundancy, allowing the system to continue functioning even if one disk fails.
- **Load Balancing:** Distributing network traffic across multiple servers ensures that if one server fails, others can still handle the load.
- **Clustering:** Creating clusters of servers ensures that if one server fails, another can take over the workload seamlessly.
- **Virtualization:** Running virtual machines on a server allows for easy migration of workloads to another server in case of hardware failure.
- **Microservices Architecture:** Breaking down applications into smaller, **independent services** allows for the isolation of faults, preventing the entire system from failing if one service encounters issues.
- **Distributed Cloud Architecture:** Distributing applications across multiple [cloud regions](#) or providers enhances fault tolerance by reducing the impact of a failure in a specific region or service.

Replication Strategies for Enhancing Fault Tolerance

1. Full Replication

Complete duplication of system or data across multiple nodes.

- **Advantages of Full Replication:** The system ensures straightforward **fault tolerance** with a seamless switch to a backup node in case of failure.
- **Challenges of Full Replication:** Hosting a full replica on each node makes the system resource-intensive, highlighting the importance of [synchronization](#) mechanisms for maintaining [consistency](#).

Implementation: Every node maintains an identical copy of the entire system or dataset. Read out more in [detail](#).

Replication Strategies for Enhancing Fault Tolerance

1. Full Replication

Complete duplication of system or data across multiple nodes.

- **Advantages of Full Replication:** The system ensures straightforward **fault tolerance** with a seamless switch to a backup node in case of failure.
- **Challenges of Full Replication:** Hosting a full replica on each node makes the system resource-intensive, highlighting the importance of [synchronization](#) mechanisms for maintaining [consistency](#).

Implementation: Every node maintains an identical copy of the entire system or dataset. Read out more in [detail](#).

2. Partial Replication

Selective duplication of critical components or data.

- **Advantages of Partial Replication:** Partial replication enhances resource efficiency by focusing on **replicating** only key components, which necessitates a careful selection process to determine which components are most critical for replication.
- **Challenges of Partial Replication:** Determining which parts are critical introduces complexity, and selectively replicated components present **synchronization challenges**.

Implementation: Replicates only essential elements for system functionality, optimizing resource usage.

3. Shadowing or Passive Replication

Maintaining passive copies that activate only upon primary system failure.

- **Advantages of Shadowing or Passive Replication:** Shadowing or passive replication offers resource efficiency during normal operation and ensures a quick response in case of a failure.
- **Challenges of Shadowing or Passive Replication:** Ensuring synchronization during the transition from passive to active state is essential, and having effective fault detection mechanisms is crucial for maintaining system reliability.

Implementation: Inactive replicas become active when the primary system encounters a fault.

Fault Tolerance vs. High Availability Load Balancing

Below are the differences between [Fault Tolerance](#) and [High Availability](#) Load Balancing:

Fault Tolerance

- **Definition:** Ensures a system continues to operate properly even if some components fail.
- **Primary Goal:** Maintain system functionality despite failures.
- **Key Techniques:** Redundancy, replication, failover mechanisms, error detection, and correction.
- **Redundancy:** High level of redundancy (multiple components performing the same task).
- **Examples:** RAID (Redundant Array of Independent Disks), **Distributed databases** with replication.
- **Impact on Performance:** May slightly impact performance due to redundancy checks and error handling.

High Availability Load Balancing

- **Definition:** Distributes workloads across multiple servers to ensure no single server becomes a bottleneck, ensuring system availability.
- **Primary Goal:** Maximize uptime and resource utilization by balancing load.
- **Key Techniques:** [Load distribution algorithms](#) (round-robin, least connections, etc.), health checks, failover.
- **Redundancy:** Moderate redundancy (enough to balance the load and ensure availability).
- **Examples:** DNS load balancing, Application load balancers (like [NGINX](#), HAProxy).
- **Impact on Performance:** Generally improves performance by distributing workload evenly.

Challenges in Implementing Fault Tolerance

- **Scalability Issues:** Scalability refers to the ability of a system to handle increasing workload or data size gracefully without sacrificing performance or availability. Scalability challenges in fault tolerance involve ensuring that fault-tolerant mechanisms can scale alongside the system's growth.
- **Performance Impacts:** Fault tolerance mechanisms, such as redundancy and error correction, can impact system performance. This challenge involves minimizing performance degradation while maintaining high fault tolerance.
- **Cost Considerations:** Implementing robust fault tolerance strategies often incurs additional costs due to the need for redundant hardware, software licenses, maintenance, and monitoring systems.

I. Introduction

A. Definition of High Availability, Fault Tolerance, and Disaster Recovery

1. **High Availability (HA):**
High Availability refers to the design and implementation of IT systems and infrastructure that are consistently operational with minimal downtime. The goal is to ensure that services are available even during hardware or software failures by employing redundancy, failover mechanisms, and load balancing.
2. **Fault Tolerance (FT):**
Fault Tolerance is the ability of a system to continue functioning correctly even when one or more components fail. Fault-tolerant systems are designed to detect failures and automatically correct them or bypass the malfunctioning component, ensuring uninterrupted service.
3. **Disaster Recovery (DR):**
Disaster Recovery involves strategies and measures that enable an organization to recover and restore its systems, data, and applications after a catastrophic event, such as natural

disasters, cyberattacks, or system failures. DR plans often include backup systems, offsite data storage, and predefined recovery procedures.

B. Importance of These Strategies in Modern IT Infrastructures

1. **Ensuring Service Continuity:**
In today's fast-paced digital world, businesses depend heavily on continuous access to systems and data. HA and FT strategies help organizations minimize downtime and maintain critical services without disruption, even in the event of hardware or software failures.
2. **Reducing Risks and Losses:**
Fault Tolerance ensures that systems remain operational despite unexpected failures, reducing the risk of service interruptions and ensuring a seamless experience for users. DR strategies play a crucial role in reducing the impact of catastrophic events on business operations, protecting against data loss, and preventing long-term disruptions.
3. **Competitive Advantage:**
Organizations with effective HA, FT, and DR strategies can respond faster to failures or disasters, providing them with a competitive edge by minimizing downtime and maintaining a stable service environment. This also builds trust with customers who expect reliability.

C. Overview of Their Role in Business Continuity, Data Protection, and Minimizing Downtime

1. **Business Continuity:**

Business continuity refers to the ability of an organization to continue operating its critical functions during and after a disruption. HA, FT, and DR are integral to business continuity plans, enabling businesses to maintain essential services even when faced with infrastructure failures or disasters. These strategies ensure that employees, customers, and stakeholders have access to the necessary systems at all times.

2. **Data Protection:**

Data is one of the most valuable assets for any business, and its protection is a top priority. DR strategies, such as data backups, encryption, and offsite storage, safeguard against data loss during a disaster. FT systems are designed to protect against corruption or loss of data by ensuring systems continue operating correctly during failure events.

3. **Minimizing Downtime:**

HA and FT strategies help minimize downtime by preventing service disruptions. In the event of a failure, HA systems redirect traffic to operational resources, while FT systems continue to function as intended. DR strategies, on the other hand, provide the necessary protocols to recover and restore systems quickly, ensuring minimal downtime during catastrophic events.



Together, these strategies play a vital role in maintaining the integrity, availability, and security of IT systems, ensuring that businesses can function smoothly and recover rapidly in the face of any unexpected disruptions.

II. High Availability (HA)

A. Definition and Principles

High Availability (HA) refers to the design and deployment of systems, applications, and services that are consistently available with minimal downtime. The primary goal of HA is to reduce service disruptions by ensuring that if a component fails, another can quickly take over, maintaining the system's operational state. The key principles of HA involve:

1. **Redundancy:** Having multiple copies of critical components or systems to ensure continued operation in the event of a failure.
2. **Resilience:** Designing systems to be resilient to faults by implementing fault detection, automatic failover, and self-healing capabilities.
3. **Failover:** The ability to switch to a backup system seamlessly when the primary system fails, preventing downtime.

HA is particularly important for mission-critical systems that require continuous uptime, such as financial services, healthcare systems, and e-commerce platforms.

B. Key Techniques for Implementing High Availability

1. Redundancy

Redundancy involves duplicating critical components, resources, or services to ensure system availability if one component fails. The types of redundancy include:

- **Hardware Redundancy:** Using multiple servers, power supplies, network connections, and storage devices to prevent hardware failure from causing a system outage.
- **Software Redundancy:** Running multiple instances of software applications on different servers or nodes to ensure availability in case of an application failure.
- **Data Redundancy:** Storing multiple copies of data across various locations (e.g., in different data centers or cloud regions) to prevent data loss if one storage location fails.

2. Load Balancing

Load balancing distributes incoming network traffic or workload evenly across multiple servers or systems to prevent overloading a single resource. The benefits of load balancing include:

2. Load Balancing

Load balancing distributes incoming network traffic or workload evenly across multiple servers or systems to prevent overloading a single resource. The benefits of load balancing include:

-
- **Scalability:** Load balancing helps scale services by distributing traffic across multiple servers, ensuring that the system can handle increasing workloads.
 - **Failover:** If one server becomes unavailable, the load balancer can redirect traffic to healthy servers, ensuring continued availability.
 - **Performance:** By balancing the load, the system performs optimally, preventing delays or service degradation.

Load balancing can be done at different levels, such as:

- **Layer 4 (Transport Layer):** Balancing based on IP address and port.
- **Layer 7 (Application Layer):** Balancing based on content such as HTTP requests or URL parameters.

3. Failover Mechanisms

Failover refers to the process of automatically switching to a backup system, server, or component when the primary one fails. Key elements of failover mechanisms include:

- **Automatic Failover:** In a failover system, if the primary server or system experiences a failure, the system automatically switches to a secondary or standby server without manual intervention.
- **Health Checks:** Regularly monitoring the health of critical components to detect failures proactively and trigger failover when necessary.
- **Replication and Synchronization:** Ensuring data or state information is replicated in real-time or near-real-time to the backup system, enabling it to take over with minimal disruption.

C. High Availability Architectures

1. Active-Passive Architecture

In an active-passive HA architecture, only one instance of a system or service is active and handling traffic at any given time, while the other instance remains passive or on standby. The passive instance is fully synchronized with the active instance but does not process requests unless the active instance fails.

- **Advantages:**

- Simplicity in setup and management.
- Cost-effective for low to medium-traffic applications.

- **Disadvantages:**

- Passive systems are underutilized during normal operation.
- Failover times may vary depending on the architecture.

When the active system fails, the passive system automatically takes over the workload, minimizing downtime and ensuring continuity.

2. Active-Active Architecture

In an active-active HA architecture, multiple instances of a system or service are active and share the load equally, ensuring no single point of failure. All instances handle traffic simultaneously, and if one instance fails, the remaining instances continue operating without major disruptions.

- **Advantages:**

- Full utilization of all system resources.
- Improved performance and load distribution.
- No idle resources, maximizing capacity during normal operations.

- **Disadvantages:**

- More complex to configure and maintain.
- Higher costs due to the need for multiple active systems running concurrently.

Active-active architectures provide continuous availability and are typically used for high-demand systems, such as large-scale e-commerce platforms, cloud services, and financial trading applications.

III. Fault Tolerance (FT)

A. Definition and Principles

Fault Tolerance (FT) refers to the ability of a system to continue operating correctly even in the event of a component failure. A fault-tolerant system is designed to detect and recover from failures without significantly affecting the system's performance or causing downtime. The primary principles of fault tolerance include:

1. **Redundancy:** Critical components or systems are duplicated to ensure that if one part fails, the other can take over without affecting the system's operation.
2. **Graceful Degradation:** Instead of a complete system failure, the system should continue operating at a reduced capacity in the event of a fault.
3. **Self-Healing:** Fault-tolerant systems are capable of detecting faults and automatically recovering, often through error correction or failover mechanisms.
4. **Isolation:** Faults should be contained in a way that they do not propagate through the entire system, allowing unaffected parts to continue functioning.

Fault tolerance is critical for applications that require continuous operation, such as mission-critical systems in healthcare, finance, telecommunications, and manufacturing.

B. Key Techniques for Achieving Fault Tolerance

1. Error Detection and Correction

Error detection and correction techniques allow systems to identify discrepancies in data or operation, and then automatically correct or compensate for the detected errors.

Common techniques include:

- **Parity Checks:** Adding an extra bit to data to detect errors in transmission or storage.
- **Checksums:** A value computed from data to verify its integrity during storage or transmission.
- **Error Correction Codes (ECC):** Advanced algorithms that allow the detection and correction of errors in data, especially used in memory systems and storage devices.
- **Redundant Arrays of Independent Disks (RAID):** Employing RAID configurations to detect and correct errors in disk storage by using techniques like mirroring (RAID 1) or parity-based redundancy (RAID 5).

2. Redundant Hardware and Software Systems

Redundancy is a key method of fault tolerance. By duplicating hardware or software systems, the failure of one component will not result in a total system failure.

Redundancy can be implemented in various ways:

- **Hardware Redundancy:** Using multiple servers, network components, and storage devices that are identical and synchronized to ensure that, if one fails, another can take over.
- **Software Redundancy:** Running duplicate instances of applications or services on different machines or containers, where a failure in one instance will not affect the availability of the application.
- **Network Redundancy:** Multiple network connections, switches, and routers are deployed to prevent a single point of failure from disrupting network services. Load balancers also play a role in distributing traffic across redundant servers.